

Module 3: Lesson 3

Deep Convolutional Neural Networks: Encoding Time Series as Images



Outline

- ▶ From time series to images
- ▶ Gramian Angular Field (GAF) imaging
- ▶ A simple example of GAF

From time series to images

The success of Deep Learning largely arises due to the advances in computer vision and speech recognition.

Time series forecasting with explicitly designed models can be challenging.

- ▶ We'll cover Deep Learning architecture for time series in Modules 4 and 5 of this course.
- ▶ We cannot take advantage of pre-trained models.

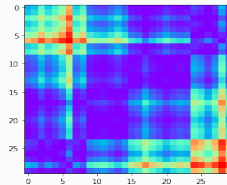
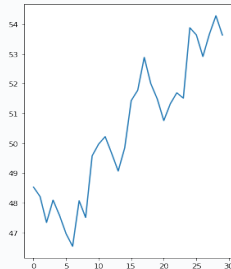
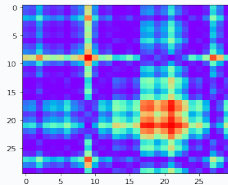
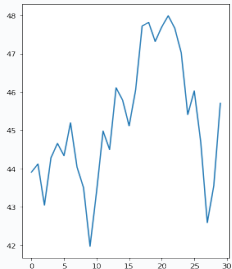
A shortcut to improving the performance of time series models may be to use convolutional neural networks.

But to do this, we need to develop a tool that transforms time series information into an image-like structure.

Here, we describe one way to encode time series as images: **Gramian Angular Field**.

Gramian Angular Field Imaging

With the Gramian Angular Field, we'll be able to represent a simple time series as an image that represents the features of the time series.



Math for Gramian Angular Field: Gram Matrix

The gram matrix is used to compute the extent of linear dependence between a set of vectors.

The gram matrix, G , of a set of n Euclidean vectors $\{v_1, \dots, v_n\}$, $v_i \in \mathbb{R}^k$, is defined by the dot-product of every pair of vectors in the set:

$$G = \begin{pmatrix} \langle v_1, v_1 \rangle & \langle v_1, v_2 \rangle & \cdots & \langle v_1, v_n \rangle \\ \langle v_2, v_1 \rangle & \langle v_2, v_2 \rangle & \cdots & \langle v_2, v_n \rangle \\ \vdots & \vdots & \ddots & \vdots \\ \langle v_n, v_1 \rangle & \langle v_n, v_2 \rangle & \cdots & \langle v_n, v_n \rangle \end{pmatrix}$$

$\langle v_i, v_j \rangle$ is the inner product of v_i and v_j : $\langle v_i, v_j \rangle = v_i \cdot v_j = \|v_i\| \cdot \|v_j\| \cdot \cos(\theta_{i,j})$. $\theta_{i,j}$ is the angle between v_i and v_j .

If the vectors have all a (Euclidean) norm of one, $\|v_i\| = \|v_j\| = 1$, then we obtain a matrix of angles between vectors $\theta_{i,j}$.

Gramian Angular Field (GAF)

The standard gram matrix captures linear correlations between elements in a time series and may fail to capture richer representations of the latent states that drive the series.

Wang and Oates (2015) suggest an encoding method from a polar coordinates transformation using the following two steps:

1. Normalize a time series $\{x_1, \dots, x_T\}$ between -1 and 1 (Min-Max scaler). Denote by $\tilde{x}_t$ the re-scaled version of observation x_t .
2. Convert the normalized data into “polar coordinates”: $\phi_t = \arccos(\tilde{x}_t)$.

We then build a gram-like matrix:

$$G = \begin{pmatrix} \cos(\phi_1 + \phi_1) & \cos(\phi_1 + \phi_2) & \cdots & \cos(\phi_1 + \phi_T) \\ \cos(\phi_2 + \phi_1) & \cos(\phi_2 + \phi_2) & \cdots & \cos(\phi_2 + \phi_T) \\ \vdots & \vdots & \ddots & \vdots \\ \cos(\phi_T + \phi_1) & \cos(\phi_T + \phi_2) & \cdots & \cos(\phi_T + \phi_T) \end{pmatrix}$$

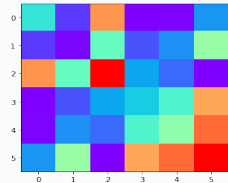
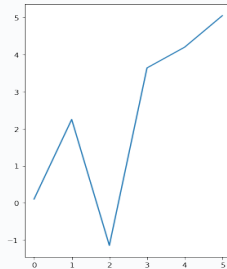
which will serve as a 2-dimensional version of the time series.

Simple example of GAF

The GAF representation of the time series penalizes those pairs of observations that are close to the mean.

Consider the time series $\{0.1, 2.25, -1.15, 3.64, 4.2, 5.05\}$. We can perform the GAF transformation as we show below:

```
# Array
X = np.array([0.1, 2.25, -1.15, 3.64, 4.2, 5.05])
# Min-Max scaling
X_norm = (X - np.min(X))/(np.max(X)-np.min(X))
X_norm = X_norm * 2 - 1
# Polar transformation
X_arccos = np.arccos(X_norm)
X_arccos_m = np.add.outer(X_arccos, X_arccos)
GramAngular = np.cos(X_arccos_m)
```



```
fig, ax = plt.subplots(1, 2, figsize=(15,7), gridspec_kw={'width_ratios': [1, 1]})
ax[0].plot(X)
ax[1].imshow(GramAngular, cmap='rainbow')
```

Summary of Lesson 3

In Lesson 3, we have:

- ▶ Shown that time series prediction problems can be transformed into image recognition tasks using convolutional neural networks.
- ▶ Described Gramian Angular Field as a tool to transform time series into images.

⇒ **References for this lesson:**

Wang, Zhiguang, and Tim Oates. "Encoding Time Series as Images for Visual Inspection and Classification Using Tiled Convolutional Neural Networks." In *Workshops at the Twenty-Ninth AAAI Conference on Artificial Intelligence*. 2015.

de Vitry, Louis. "Encoding Time Series as Images." *Medium*. October 2018.

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement the GAF imaging for financial data.